3.8 Minimum Cost Spanning Tree

3.8.1 Graph

Graph (V., E)

↙ ↘

Set of vertices set of edges

- Let G(V, E) be a simple graph then

Maximum edges = $\frac{V(V-1)}{2}$

$E \leq \frac{V(V-1)}{2}$

$E \leq C.V^2$ C is constant

Note:

$E = O(V^2)$

$\text{Log } E = O(\log V)$

3.8.2 Graph Representation

Graph Representation

↙ ↘

Adjacency matrix Adjacency list

- For more edges (Dense Graph) Adj. matrix is better (density more).
- For less edge (sparse graph) Adj list is better.

	Matrix	List
(1) Finding degree of vertex⇒ Time Complexity	O(V) Every Case	O(1) Best Case O(V1) Worst Case
(2)Finding total edges ⇒ Time Complexity	$O(V^2)$ Every Case	O(V+2E) Worst Case O(V) Best Case
(3) Finding 2-vertices adjacent (or)not ⇒ Time Complexity	O(1)	O(V-1) Worst Case O(1) Best Case
(4) G(V,E) ⇒ space	$O(V^2)$ Every Case	O(V+E) Every Case

3.8.3 What is Spanning Tree

A subgraph T(V, E’) of G(V, E) where E’ is the subset of ($E' \subseteq E$) is a spanning tree iff ‘T’ is a tree.

A sub graph G(‘V, E’) of G(V, E) is said to be spanning tree.

(1) T’ should contain all vertices of G

(2) T’ should contain (V-1) edged where V is number of vertices without cycle.

(3) T' should connected.

3.8.4 Minimum Cost Spanning Tree

Minimum cost spanning tree is the one in which cost of the spanning tree formed should be minimum.

3.8.5 Prims Algorithm

- Select Any vertex

  Time complexity = V + VlogV + 2E + ElogV

  = O(E + V)logV

Using Sorted Array & Adjacency List

V + 2E + E × V = O(EV)

Using Sorted Array & Adjacency List

V × O(1) + V2 + E × V= O(EV)

3.8.6 Kruskal algorithm

- Take first minimum edge

  Time complexity = E log E + (V + E)

  = O(E log E) = O(ElogV)

  If edges are already sorted

  TC = O(E + V)

3.9 Single Source Shortest Path

3.9.1 Dijkstra Algorithm

- Using min heap & adjacency list = O(E + V)logV
- Using adjacency Matrix & Min heap = O(V^2 ElogV)
- Using adjacency list & Unsorted Array = O(V^2)
- Using adjacency list & Sorted Doubly Linked List = O(EV)

3.9.2 Bellman-Ford

- Time Complexity = O(EV)
- If negative edge weight cycle then for some vertices Incorrect answer.

❑❑❑